



Tecnológico de Monterrey

Actividad: Momento de Retroalimentación: Módulo 2
Implementación de una técnica de aprendizaje máquina
sin el uso de un framework.

Adrián Proaño Bernal A01752615

2 de Septiembre del 2026

Inteligencia artificial avanzada para la ciencia de datos (Gpo 601)

Profesor: Jorge Adolfo Ramírez Uresti

Introducción:

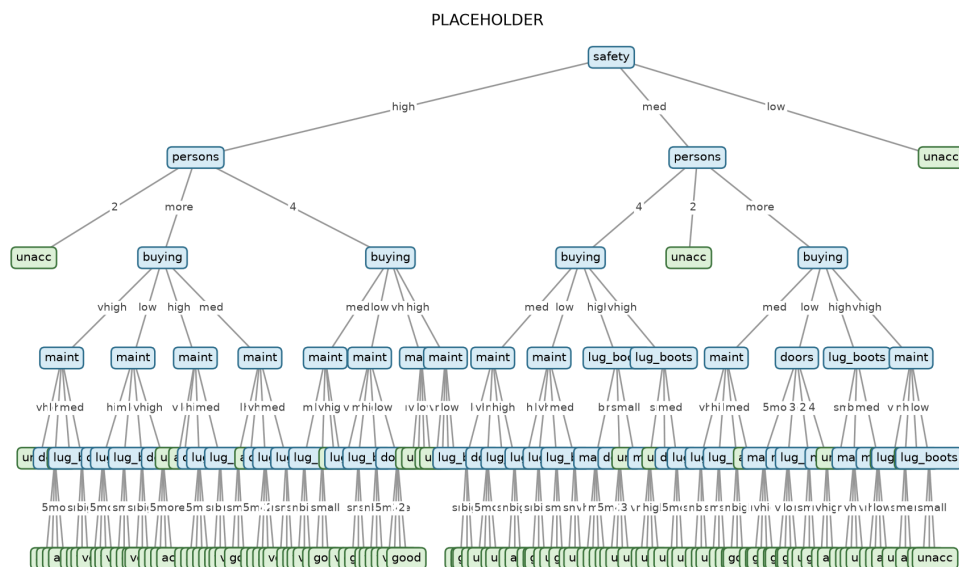
En esta actividad, se nos pide escoger un algoritmo de aprendizaje de inteligencia artificial y generarlo en código sin utilizar un framework existente de machine learning. Escogí la implementación de árbol de decisiones utilizando el algoritmo ID3, el cual se utiliza como generador de árbol de decisiones creado por Ross Quinlan. La idea principal era entender y reproducir las mecánicas detrás de cómo el árbol de decisiones decide qué features seleccionar primero, y cuales descartar como irrelevantes.

Este algoritmo fue evaluado en el “Car Evaluation dataset” del repositorio de UCI Machine Learning. Este dataset contiene 1728 instancias, cada una de las cuales describe un coche utilizando 6 atributos categóricos: “Buying price”, “Maintenance cost”, “Number of doors”, “Passenger capacity”, “Luggage”, “Boot size” y “Safety rating”, con la meta de predecir qué tan aceptable es un coche (“unacceptable”, “acceptable”, “good”, “very good”). Este dataset teniendo puros atributos categóricos lo hacen una muy buena elección para el algoritmo de ID3.

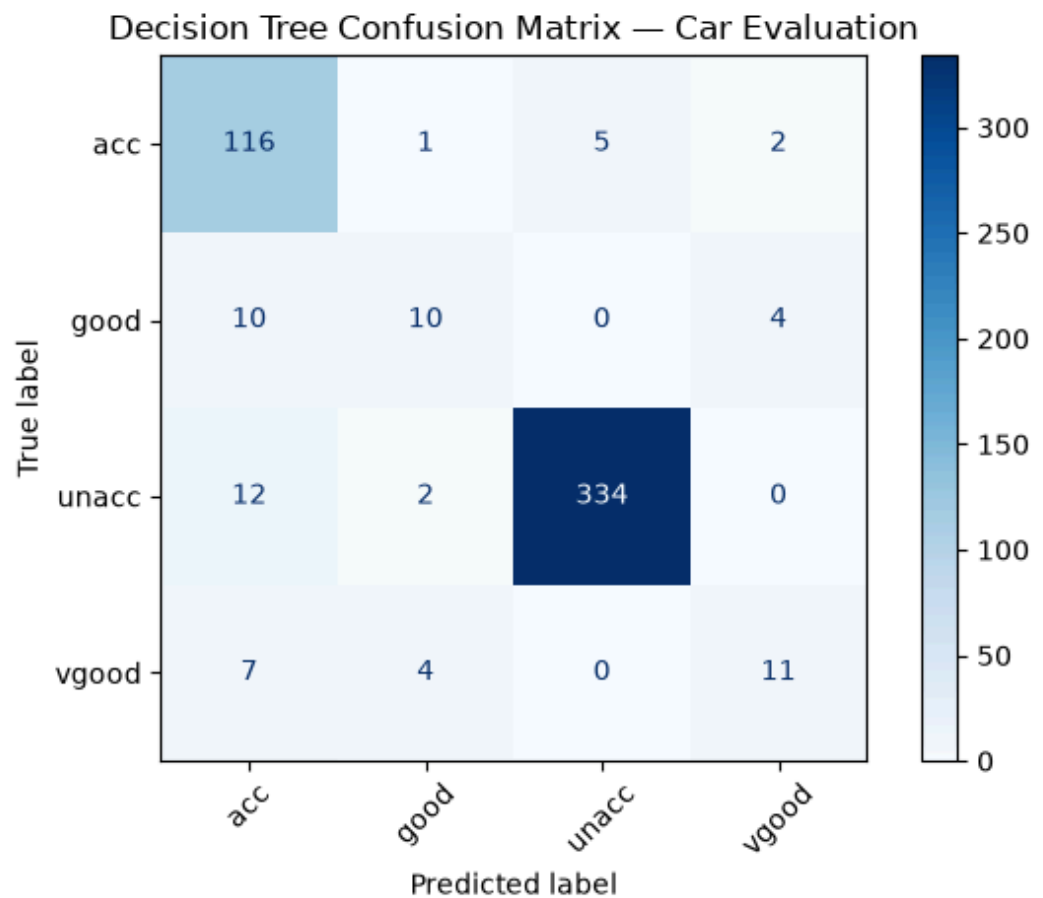
En cada paso de, el algoritmo seleccionara el atributo que de él “information gain” más alto, esté “information gain”, siendo simplemente un análisis de qué tan “útil” un atributo es, en el sentido de que tanto ayuda a separar los datos en grupos basándose en los posibles casos de los atributos. El algoritmo calcula el “score” de cada atributo y escoge el atributo que hace el mejor trabajo en distribuir los datos y después, hace eso consecutivamente con cada otro atributo. El árbol es construido recursivamente, cada nodo interno representando un atributo y cada rama, representando los posibles valores de ese atributo, continuando esto hasta que todos los datos de un nodo pertenezcan a una sola clase, que no queden más atributos para seguir dividiendo datos o que se haya llegado a la máxima profundidad asignada al árbol. Para evaluar el modelo resultante, el dataset se dividió en dos subsets, “training” y “testing”, y al final, las predicciones del árbol con los datos no vistos fueron directamente comparados con los datos reales.

Resultados:

1. Visualización de árbol



2. Matriz de confusión:



3. Precisión de Predicciones

```
OneDrive/Documentos/GitHub/-Imp
Overall Accuracy: 90.93%
Accuracy for unacc: 95.98%
Accuracy for acc: 93.55%
Accuracy for vgood: 50.00%
Accuracy for good: 41.67%
```

Análisis de resultados:

Al entrenar el árbol de decisiones con un tamaño de set del 70% del completo, y probarlo con el resto del 30%, podemos ver que en general, la precisión está bastante alta, aunque cuando analizamos la precisión por clase, podemos ver que en unacc y acc, acierta casi todo el tiempo, mientras que en vgood y good, acierta la mitad de las veces o menos.

Mis conclusiones de esto son que no es tanto que el árbol esté mal hecho, sino más bien, al analizar el dataset, podemos ver que muchísimos datos terminan clasificados en las clases de unacc y acc. Esto causa que nuestro árbol de decisiones este principalmente entrenado en escenarios en donde la mayoría de las clases acaban en unacc o acc, entonces el árbol se va principalmente por esas dos clases al intentar predecir. Si tengo planeado utilizar otro dataset de clasificación más distribuido de forma pareja.

Data set utilizado: Car Evaluation (UCI Machine Learning)

<https://www.kaggle.com/datasets/rohit265/car-evaluation-uci-machine-learning>